KawaiiTránsito

DOCUMENTO TÉCNICO

Calculadora de tránsito

Frontend modular · Trivia de cultura vial · Experiencia kawaii

DESARROLLADORAS

Nashla Fernandez · Aida Manzueta

Versión 1.0

♡Hecho con lógica y tecnología ♡

Contenido

01 1. Resumen del problema
02 2. Alcance técnico
03 3. Arquitectura
04 4. Modelo de datos
05 5. Algoritmos relevantes
06 6. Servicios y manejo de errores
07 7. Accesibilidad y experiencia de usuario
08 8. Estilos
09 9. Rendimiento
10 10. Pruebas y verificación
11 11. Riesgos y mitigaciones
12 12. Trabajo futuro

1. Resumen del problema

La calculadora de tránsito original vivía en un único archivo HTML con estilos y lógica en línea: no había separación de responsabilidades, ni capa de datos reutilizable, ni forma de verificar el código. El proyecto reemplaza ese archivo por una aplicación de módulos ES6 donde conviven dos módulos de producto:

1. **Calculadora de tránsito** — compara modos de transporte usando distancia, duración y clima reales, y produce una recomendación explicada.
2. **Trivia de cultura vial** — consume un banco de preguntas, gestiona tiempo y vidas, otorga insignias y publica el puntaje en una clasificación.

Ambos comparten la misma estructura de carpetas, el mismo sistema visual kawaii y las mismas utilidades.

2. Alcance técnico

Incluido	Excluido
Frontend completo con módulos ES6 nativos	Backend propio de la API
Integración con APIs públicas (sin cuenta)	Cuentas de usuario y login
Capa de servicios preparada para API propia	Persistencia en servidor
Persistencia local con `localStorage`	Sincronización entre equipos
Servidor estático de desarrollo en Node	Despliegue en la nube

3. Arquitectura



3.1 Capas y dependencias permitidas

Capa	Carpeta	Responsabilidad	Puede importar de

Entrada	`main.js`	Construir servicios, montar shell, registrar rutas	todas
Vistas	`views/`	Componer una pantalla completa	components, modules, services, utils, config
Componentes	`components/`	Crear y actualizar piezas visuales	utils, data, config
Módulos	`modules/`	Reglas de negocio, cálculo, estado	utils, services
Servicios	`services/`	talking to the network and storage	utils, config
Utilidades	`utils/`	Funciones puras reutilizables	nada
Datos	`data/`	Contenido estático	nada
Config	`config/`	Endpoints y configuración pública	env

Reglas transversales que el proyecto respeta y verifica:

♡fetch solo aparece en services/http-client.js.

♡Ninguna vista contiene if de negocio; solo compone y coordina eventos.

♡Ningún archivo declara estado global; todo vive en clases o closures.

4. Modelo de datos

4.1 Objetos de dominio

```
Location   { name, lat, lon, shortName, isAccurate }
Trip       { mode, distanceKm, baseDurationMinutes, cost,          congestionRatio,
estimated, allTrips }
TripPlan   { origin, destination, weather, options, trips,         fastest,
preferred }
Weather    { temperature, precipitationMm, windKmh, severity,      label,
description }
TriviaQuestion { id, categoryId, prompt, answers[], correctIndex }
TriviaResult   { score, correct, total, accuracy, maxCombo, medals[] }
```

Los getters calculan en lugar de duplicar: Trip.durationMinutes aplica la congestión,

Trip.extraMinutes deriva el retraso y PlayerProfile.bestScore lee el almacenamiento.

4.2 Persistencia local

Clave kawaii-transito en localStorage:

```
{
  "player":   { "name": "Mochi", "avatar": "1F638", "bestScore": 980,
                "roundsPlayed": 6 },
  "badges":   { "unlocked": ["primera-partida", "racha-perfecta"] },
  "leaderboard": [ { "name": "Mochi", "score": 980, "accuracy": 1,
                     "createdAt": "2026-01-12T18:00:00.000Z" } ] }
```

StorageRepository encapsula localStorage y aplica el sobre

{ value, updatedAt } junto con un try/catch por si el navegador bloquea el almacenamiento.

4.3 Esquema propuesto para la API propia

Aunque el alcance actual no incluye backend, esta es la estructura que el frontend espera y que el documento técnico deja definida:

```
players
 ├─ id          PK
 ├─ name        UNIQUE, 3..24 caracteres
 ├─ avatar      VARCHAR(8)
 ├─ best_score  INT DEFAULT 0
 ├─ games_played  INT DEFAULT 0 └─
created_at   TIMESTAMP

scores
 ├─ id        PK
 ├─ player_id   FK -> players.id
 ├─ score     INT
 ├─ correct   INT
 ├─ total     INT
 ├─  accuracy       DECIMAL(4,3)  └─
played_at   TIMESTAMP

badges
 ├─ id       PK
 ├─ code     UNIQUE
 ├─ label    VARCHAR(40)
 └─ threshold  INT
```

Consultas previstas: GET /api/leaderboard?limit=10 ordena por score y, a empate, por accuracy; POST /api/leaderboard inserta una partida y devuelve el ranking actualizado.

5. Algoritmos relevantes

5.1 Perfil de congestión

El servicio público de OSRM no entrega tráfico en tiempo real, así que el proyecto usa un **perfil horario** explícito y documentado (modules/traffic-profile.js): una tabla de 24 valores con la ocupación vial típica de un día laboral (0.05 de madrugada, 0.85 entre las 08:00 y las 09:00, 0.22 a mediodía). El clima del destino aplica una penalización adicional según su severidad.

```
congestionRatio = hourlyProfile[hour] + weatherPenalty(weather.severity) durationMinutes =
baseDurationMinutes * (1 + congestionRatio)
```

Si la API de rutas no responde, la distancia se obtiene con la fórmula de Haversine sobre las coordenadas geocodificadas y la duración se estima con la velocidad media del modo; esos

viajes se marcan como estimated y la interfaz avisa al usuario con una alerta visible, nunca de forma silenciosa.

5.2 Comparación de modos

TripPlanner normaliza las estimaciones para que bicicleta y a pie no aparezcan con rutas desproporcionadamente largas, aplica la penalización fija del bus y finalmente ordena por duración corregida. RecommendationEngine produce consejos con prioridad, y cada consejo declara su motivo para que el usuario entienda por qué el sistema recomienda lo que recomienda.

5.3 Ronda de trivia

```
startRound  -> baraja preguntas y toma hasta 8
answer(i)   -> compara con correctIndex, descuenta vida si falla,          suma
racha y puntos
timeout     -> cuenta como fallo y consume una vida next()
-> aplica bono final y cierra la ronda finish()    -> calcula
medallas, insignias y clasificación
```

6. Servicios y manejo de errores

http-client.js centraliza todo el acceso de red:

- ♡compone la URL respetando direcciones absolutas (las APIs públicas ya lo son);
- ♡serializa parámetros y añade tiempo de espera con AbortController;
- ♡propaga una señal externa para cancelar peticiones al recalcular;
- ♡desenvuelve el sobre { success, data } y normaliza cualquier fallo a un ApiError con code estable (TIMEOUT, NETWORK_ERROR, HTTP_ERROR). Cada vista decide cómo mostrar el error: la calculadora usa alertas y avisos temporales, la trivia muestra un panel de reintento y la clasificación degrada a los datos locales sin interrumpir la navegación.

7. Accesibilidad y experiencia de usuario

- ♡Navegación por teclado en todos los controles; tabindex="-1" en el main para que el foco viaje a la vista nueva tras cambiar de ruta.
- ♡Roles ARIA en barra de progreso, insignia de tiempo y región de avisos (aria-live="polite").
- ♡El color nunca es el único indicador: los estados del semáforo, las vidas (♥/♡) y las respuestas correctas llevan además texto o clase.
- ♡prefers-reduced-motion desactiva animaciones de carga y pulso.
- ♡Diseño responsive con puntos de quiebre en 900 px y 620 px.

8. Estilos

Nueve hojas separadas por responsabilidad, todas en src/css/, sin <style> ni atributos style en el HTML:

Archivo	Contenido
`base.css`	Variables, reset, tipografía, botones de foco
`layout.css`	Shell, encabezado, navegación, pie, contenedores
`components.css`	Tarjetas, botones, campos, insignias, medidor
`data-display.css`	Tablas de datos
`feedback.css`	Avisos, alertas, carga, estados vacíos
`calculator.css`	Módulo 1
`trivia.css`	Módulo 2: tablero, tarjetas y respuestas
`trivia-summary.css`	Módulo 2: categorías, resumen e insignias
`leaderboard.css`	Clasificación y podio

El color se define una sola vez como variables CSS; ningún componente repite valores hexadecimales fuera de los avisos de estado, que usan pares borde/fondo/texto para mantener el contraste accesible.

9. Rendimiento

♡El grafo tiene 51 módulos y no requiere compilación: el navegador cachea cada archivo por separado.

♡Las hojas de estilo se piden en paralelo desde index.html.

♡Las preguntas se barajan una vez por ronda, no en cada respuesta.

♡El cliente HTTP cancela las peticiones anteriores al recalcular un viaje, lo que evita respuestas fuera de orden.

♡El mapa se incrusta con iframe de OpenStreetMap bajo demanda: no se crea si el usuario no llega a la vista de resultados.

10. Pruebas y verificación

El proyecto incorpora tres capas de verificación:

1. tools/check-conventions.mjs — reglas del enunciado sin dependencias.
2. ESLint con no-magic-numbers, límites de líneas y profundidad.
3. Comprobación de enlace de módulos: los 51 archivos deben enlazar sin erroresde exportación.

```
npm run verify
```

11. Riesgos y mitigaciones

Riesgo	Mitigación
Nominatim limita el uso sin clave	Una petición por búsqueda, `User-Agent` identificable y caché local de la última consulta
OSRM público intermitente	Estimación por Haversine marcada como `estimated` y avisada al usuario
Open-Meteo sin disponibilidad	El clima es opcional: el plan se calcula igual
API propia ausente	Respaldo automático con datos locales
Almacenamiento bloqueado en modo privado	`try/catch` y estado en memoria
Preguntas repetidas entre rondas	Barajado con `crypto.getRandomValues`

12. Trabajo futuro

- ♡Backend Node/Express con los esquemas de la sección 4.3.
- ♡Autenticación y cuentas de jugador.
- ♡Catálogo de preguntas administrable desde la API.
- ♡Pruebas unitarias con el sistema de módulos nativo de Node y cobertura.
- ♡Estadísticas de jugador yetable por categoría.